oom-watch — Architecture and Design Decisions

Why it is the way it is

OOM-Protect maintainers

2026-04-30

oom-watch — Architecture

This document explains the *why* behind the daemon. The user manual (`manuals/oom-watch-manual.md`) explains the *how*.

Single-sentence summary

`oom-watch` is an idempotent, stdlib-only Go daemon that periodically asks atop for one parseable sample, runs it through a pure threshold engine, and — if the verdict is non-zero and the cooldown has elapsed (or an escalation has occurred) — atomically writes a Markdown forensic report assembled from the sample plus best-effort /proc, cgroup, and journal captures.

Component diagram

Why these decisions

Why Go and not bash

`oom-hardening.sh` and `oom-runner.sh` are bash because they are *one-shot installers / wrappers* — they need to be reviewable as plain text by a sysadmin. `oom-watch` is *long-running and stateful* (cooldowns, severity tracking) and must run without operator attention for weeks. Bash doesn't give you typed structs, a stdlib HTTP/JSON parser that won't panic, or `slog`. Go does, and produces a single statically-built binary.

Why atop and not /proc directly

Three reasons. First, atop already does the hard work of consistent sampling, command-name handling (the `(cmd with spaces)` form), and per-process accounting (PRM, PRC). Re-implementing that would be a multi-month project with many subtle bugs. Second, atop is already mandatory in this environment because the post-mortem (`reports/Crash_Report.md`) recommends it. Third, atop's parseable output (`-PALL`) is stable and documented in the man page; we depend on a contract atop maintains, not on internals.

Why stdlib only (no yaml.v3, no testify)

Reproducibility. The binary must build offline, on any machine that has Go 1.22+, with no `go mod tidy` network call. JSON is universal; sysadmins are comfortable editing it. testify is convenient but every assertion it provides can be written in 5 lines of stdlib `t.Errorf`.

Why JSON config and not YAML

JSON has zero ambiguity (no Norway problem, no significant whitespace), is in the stdlib, and the full config is < 30 lines. YAML's only advantage — comments — is replaced here by aliased keys (`_comment`) plus the example file being heavily commented.

Why an `OnIncident` callback instead of monitor importing snapshot

Acyclic dependencies. `snapshot.Snapshot` has a `Verdict` field of type `monitor.Verdict`. If `monitor` then imported `snapshot` to call `Capture`, we'd have a cycle. The cleanest fix is to make `monitor` declare the *callback shape* and have `cmd/oomwatch/main.go` wire `snapshot.Capture` and `report.WriteMarkdown` together.

Why "best-effort" snapshot capture

A perfect-or-nothing snapshot would miss the most important reports — the ones produced under degraded conditions where /proc/pressure is unreadable or journalctl times out. A report with 8 of 10 sections + a `Capture errors` section that names the missing two is far more useful than no report at all. Tests enforce this: `TestCapture_PartialFailure` asserts the snapshot is non-nil and `Errors` lists the missing files.

Why atomic writes

If the daemon is killed mid-write (the OOM scenario itself!) we must not leave a half-written `.md` that downstream tooling will misparse. We write to `.oom-watch-*.tmp` in the same directory and `os.Rename` on success. `TestWriteMarkdown_NoLeftoverTemp` verifies this.

Why the cooldown bypasses on escalation

Without it, the most important transition — WARN → CRITICAL — would be silently dropped if it occurred within the warn-level cooldown window. That's exactly when you most need a report. `TestLoop_EscalationBypassesCooldown` verifies this.

Threshold rationale

Metric	Default	Why
`memory_used_ratio_critical`	0.95	Below this the kernel can usually reclaim from caches without thrashing. At 0.95 the next allocation is likely to swap or OOM.
`psi_mem_full_avg10_critical`	30.0	"All non-idle tasks fully stalled on memory ≥30% of the last 10 seconds" — the system is no longer making forward progress.
`psi_mem_full_avg10_warn`	10.0	Detectable user-visible slowness begins around here.
`swap_used_ratio_critical`	0.80	Beyond 80% swap, swapping latency dominates and any further pressure means OOM kills.
`load_per_cpu_critical`	4.0	Sustained 4× saturation on per-CPU load is not transient queueing; something is genuinely overloaded.

These mirror the post-mortem in `reports/Crash_Report.md` and the systemd-oomd defaults set by `oom-hardening.sh`.

Anti-bluff testing — applied

Every package has at least one test that asserts a specific value derived from a real fixture. Examples:

- `TestParse_RealFixture` asserts `MEM.AvailPages == 4500000` and `PSI.MemFullAvg10 == 45.20` — both extracted from the fixture by hand. A parser that returned zero values would fail.
- `TestCapture_FakeProc` writes specific bytes to a fake `/proc/meminfo` and asserts those bytes appear in the snapshot.
- `TestWriteMarkdown_FullReport` writes a known Snapshot, reads the file back, and asserts every required section heading appears in the body.

The mutation Challenge (`challenge-tests-are-not-bluffs.sh`) breaks `AvailPages: pi(24)` to `AvailPages: 0`, runs the suite, asserts non-zero exit, restores. If a future contributor rewrites the parser as a no-op, the Challenge will fail.

What this daemon does *not* do

- **It does not kill processes.** Killing is the job of `systemd-oomd` (configured by `oom-hardening.sh`) and the kernel OOM-killer. oom-watch only writes reports.
- **It does not page or alert externally.** No email, no Slack, no PagerDuty. Reports are local artifacts; integrate with your alerting tool of choice (an `inotifywait` watcher on `/var/log/oom-watch/reports/` is a typical first step).
- **It does not rotate or compress reports.** Use `find ... -mtime +30 -delete` or logrotate.
- **It does not run as non-root.** It needs to read /proc//stat for every process and the cgroup tree; doing that as a non-root user requires per-process permissions that are not portable.

Future work

- **netatop-bpf integration** — atop's optional `netatop` module emits PRN (per-process network) lines. Wire those into the snapshot when present.
- **Memory.events delta tracking** — read `/sys/fs/cgroup/.../memory.events` and report `oom_kill` / `oom` deltas since previous sample.
- **HTTP endpoint** — a `--listen :9999` mode that serves the latest 100 reports as a JSON index. The `atophttpd` project shows the pattern.
- **Plot generation** — convert atop logs around the incident into PNG plots, link from the report. The `atopsar-plot` project is a starting reference.

See also

- `Constitution.md` — anti-bluff testing charter (Article I).
- `manuals/oom-watch-manual.md` — user-facing operations guide.
- `reports/Crash_Report.md` — the 2026-04-28 incident this toolkit answers.

- `oom-watch/README.md` — submodule entry point.